

```
In [ ]: # PORTFOLIO #5: STRINGS
        # CONCEPT: MOLAR MASS OF MULTI-COMPONENT MIXTURE CALCULATOR
```

In chemistry, calculating for the average molar mass of a multi-component mixture in a process stream or distillation column manually is tedious and prone to human errors.

Using the string code, digital calculations may help to easily obtain the molar mass for further analysis. We can use a loop to iterate through a list of strings and their corresponding molar fractions.

```
In [6]: # Serves as the dictionary, it is used to store data value pairs
        # Allows for faster lookups, instead of searching for a list of items
        masses = {"H": 1.008, "O": 15.999, "C": 12.011, "N": 14.007}

        components = {
            # The code "masses["C"] is an example of subscript expression
            # Retrieves the specific value associated from the dictionary
            "Methane (CH4)": (masses["C"] + 4 * masses["H"]),
            "Ethane (C2H6)": (2 * masses["C"] + 6 * masses["H"]),
            "Propane (C3H8)": (3 * masses["C"] + 8 * masses["H"])
        }

        fractions = {
            "Methane (CH4)": 0.70,
            "Ethane (C2H6)": 0.20,
            "Propane (C3H8)": 0.10
        }

        # Calculates the given above
        # Allows to transform or filter data without creating a new list
        molar_mass = sum(components[name] * fractions[name] for name in components)

        print(f'{"Component":<20} | {"Fraction":<10}')
        print("-" * 35)
        for name, frac in fractions.items():
            # These are format specifiers (:<20 or :.4f)
            # For formatting syntax
            print(f'{"name":<20} | {frac:<10.2f}')

        # Divides the final answer from the given components and fractions
        print("-" * 35)

        # Shows the result
        print("Molar Mass:", (round(molar_mass)), "g/mol")
```

Component	Fraction
Methane (CH4)	0.70
Ethane (C2H6)	0.20
Propane (C3H8)	0.10
Molar Mass: 22 g/mol	